

Vibe Coding and Vibe Engineering





Contents

- What is Vibe Coding?
 - Developer Workflow
 - Vibe Coding Tools
 - Best Practices
 - Risks and Benefits
- What is Vibe Engineering?
 - Principles of VE
 - Organisational Implications

What is Vibe Coding?



What is Vibe Coding?

- “Vibe coding” is the use of natural language to program via generative AI. So the coding is done by expressing intent or a “vibe,” rather than writing logic line by line.
- *“You just say things, run things, and it mostly works.”* - Andrej Karpathy, OpenAI

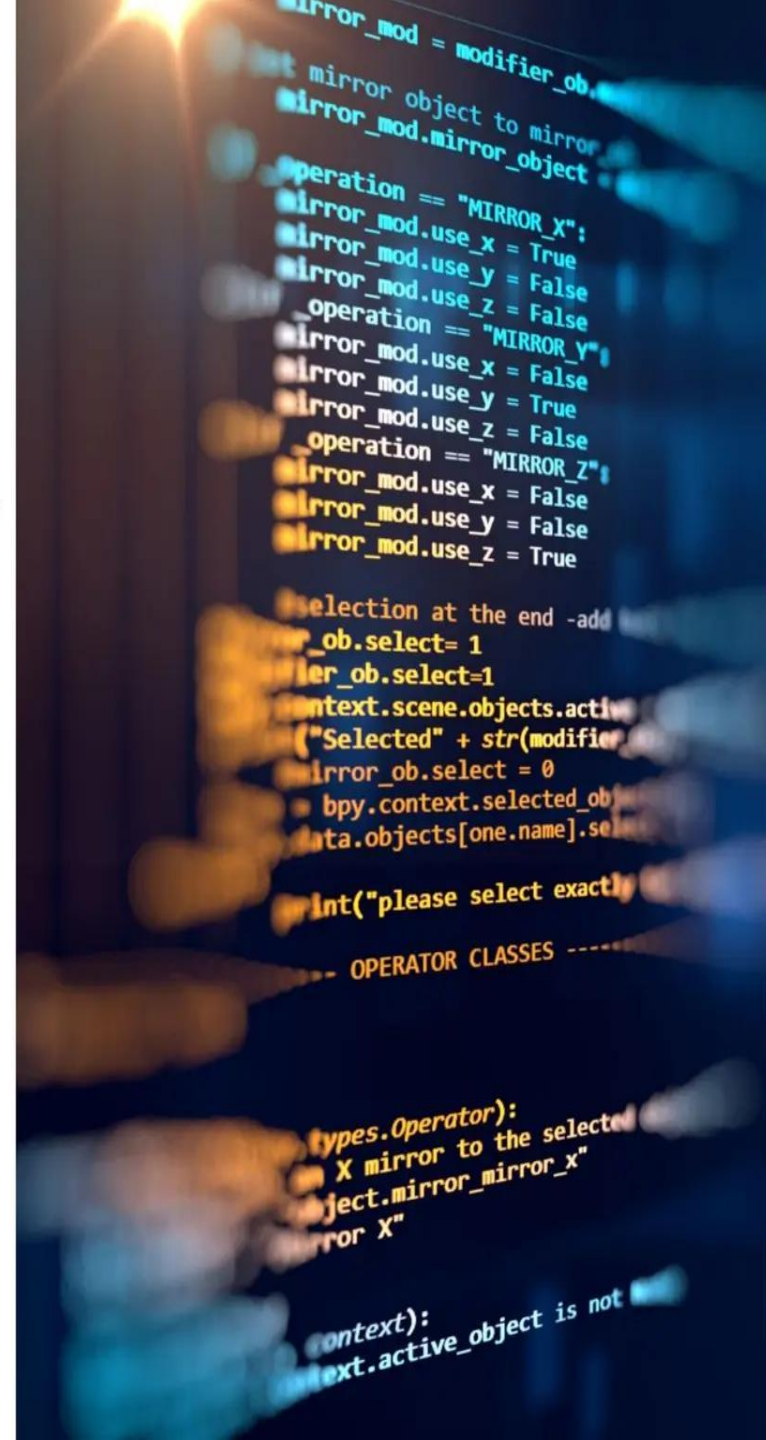
What is Vibe Coding?

- A **simple analogy** would be hiring your own personal shopper to buy clothes for you.
 - You describe the look or feel of clothes you want (e.g. "*I need a cozy set of summer outfits*")
 - The shopper interprets your intent and curates outfits
 - You try things on, give feedback ("*Too formal*", "*Try warmer tones*")
 - The shopper handles the technical side: sizing, brands, inventory
 - The shopper speeds up your process but doesn't remove your taste or final judgment

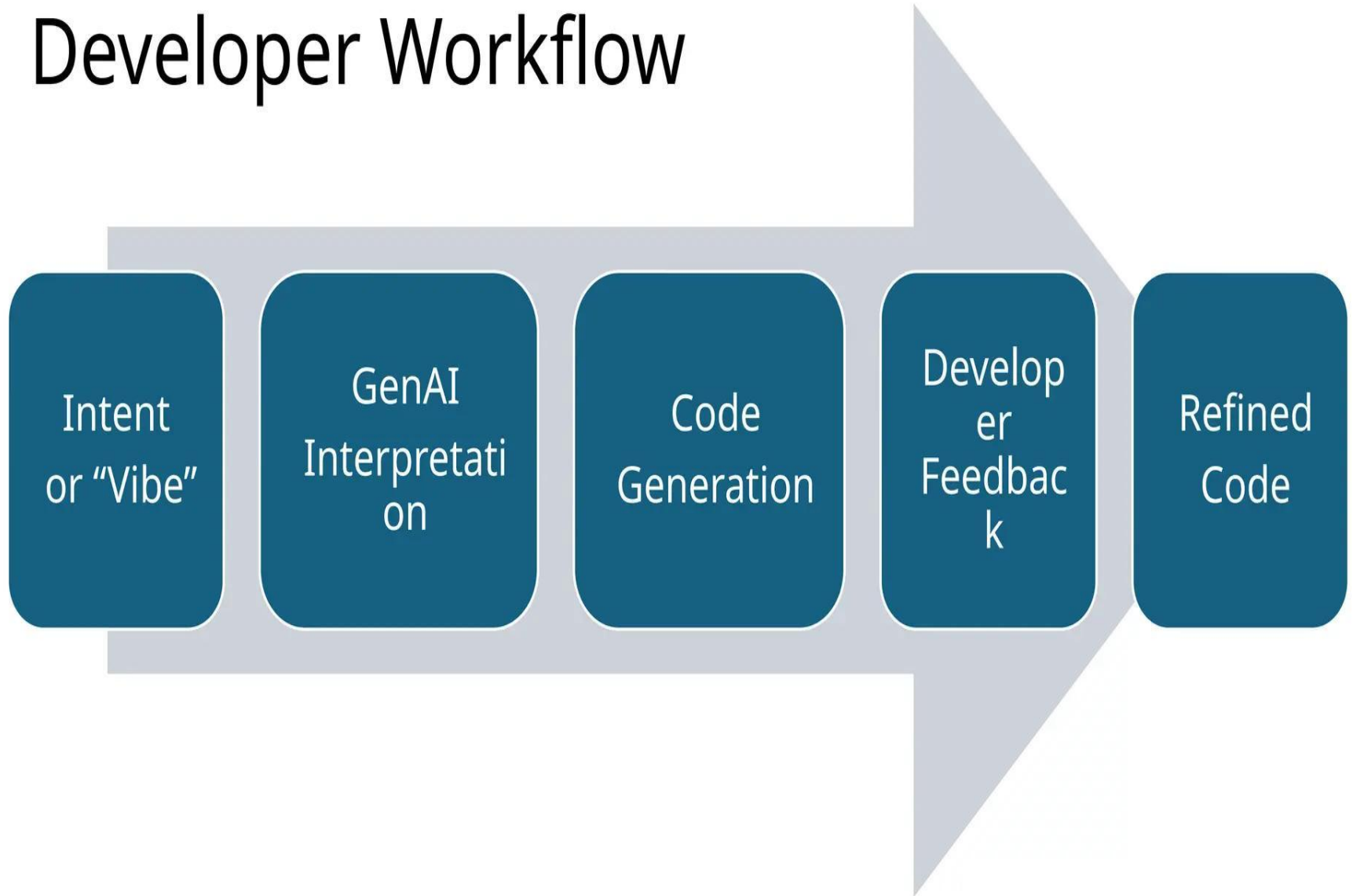


What is Vibe Coding?

- So for Vibe Coding.
 - You describe the vibe or function of the software you want (e.g, *"Build a cozy landing page with soft gradients and gentle animations"*)
 - The GenAI interprets your prompt and generates code (UI, copy, layout, animations)
 - You test/refine the output (*"Make it simpler"*, *"Use a pastel palette"*, *"Add spring easing"*)
 - The GenAI handles implementation: syntax, layout logic, libraries
 - The AI accelerates development, but you still review, edit, and deploy



Developer Workflow



Vibe Coding Tools

ChatGPT / Claude / Gemini

- Conversational coding

Cursor

- AI-native code editor

SuperWhisper

- Voice-to-code

Replit Ghostwriter

- In-browser AI dev

Lovable.so / Builder.io

- Natural language UI builders

Vibe Coding: Prompt Guide

- **Structure for Effective Prompts**

- **1. Intent:** What are you building?
- **2. Vibe:** What should it **feel** like?
- **3. Style/Constraints:** Tech stack, design choices, features
- **4. Output format:** What do you want returned?

Vibe Coding: Prompt Guide

- **Example Prompt**

- `"Create a landing page for a meditation app. The vibe should be calm and minimal with a soft gradient background and slow animations. Use React + Tailwind. Return full component code."`

Vibe Keywords

Visual	Motion	Tone	Style
soft, dreamy	smooth, slow, fluid	playful, formal	retro, brutalist
cozy, warm	springy, elastic	gentle, sharp	minimal, maximal
dark, moody	quick, responsive	calming, bold	glassy, grainy

Vibe Coding: Prompt Guide

- **Refinement Prompts**

- "Make it feel lighter/more playful."
- "Use serif fonts for a vintage vibe."
- "Add hover effects that feel soft."
- "Convert this to Vue instead of React."
- "Can you simplify the layout?"

Best Practices

Prompt Engineering is the new syntax.

Prompt structure: give context, intent, constraints.

You design by mood, outcome, or story

"Make it feel like a rainy Sunday."

Iteration is fluid

"Make it more minimal." "Try it in a darker theme."

Code vetting

The human must be in the loop.

Vibe Coding: Risks

- **Understanding:**

- The GenAI doesn't understand anything, so it's up to you understand the code even if you didn't write it.

- **Debugging bottlenecks:**

- What happens when something breaks?

- **Security issues:**

- AI might introduce silent vulnerabilities.

- **Ethical concerns:**

- Who owns the generated code?

Vibe Coding: Risks

- Contradictions
- Repetition
- Ambiguity
- Stereotyping
- Hallucinations
- Bias
- Overgeneralization
- Toxicity

Types of Errors GenAI Produces



C **Contradictions**
Conflicting or self-opposing statements

R **Repetition**
Unnecessary loops or redundant output

A **Ambiguity**
Vague or unclear generation

S **Stereotyping**
Reflecting societal bias and assumptions

H **Hallucinations**
Making things up (confidently wrong)

B **Bias**
Skewed output due to training data

O **Overgeneralization**
Applying ideas too broadly

T **Toxicity**
Harmful or offensive content

Vibe Coding: Benefits

- **Accessibility:**

- Lowers the barrier to entry for software development, enabling non-programmers to create applications.

- **Efficiency:**

- Accelerates the development process by automating routine coding tasks.

- **Creativity:**

- Allows developers to focus on high-level design and functionality

Summary

- Vibe coding is not the end of programming - it's the beginning of a new kind of creative-human/AI collaboration.

What is Vibe Engineering?



What is Vibe Engineering?

- “Vibe Engineering” is a strategic approach to software development that treats AI not merely as a tool but as a collaborative team member.
- It involves reimagining how software teams operate by integrating AI into every facet of the development lifecycle.



Vibe Coding vs Vibe Engineering

	Vibe Coding	Vibe Engineering
Focus	Individual productivity	Organizational transformation
AI Role	Assistant generating code snippets	Collaborative team member with responsibilities
Workflow	Prompt-based code generation	Structured workflows integrating AI
Quality Assurance	Manual testing and refinement	Automated verification and feedback loops
Knowledge Management	Ad-hoc context handling	Persistent memory systems for context retention

Core Principles

- **AI as a First-Class Team Member:** AI agents are assigned defined responsibilities and capabilities, collaborating alongside human developers.
- **Verification-Driven Development:** Implementing systematic testing and automated feedback loops to ensure quality and reliability.
- **Structured Workflows:** Designing architectural patterns that leverage AI's strengths while mitigating its limitations.
- **Relentless Knowledge Capture:** Maintaining context across AI interactions through persistent memory systems to preserve organizational knowledge.
- **Human Strategic Direction:** Focusing human expertise on architectural decisions and critical thinking, guiding AI contributions effectively.

Vibe Engineering Tools

Cursor, Replit Ghostwriter

- AI-native code

CodiumAI, Github Copilot

- Test generation

Windsurf

- AI context management

GitHub Copilot Enterprise, Lovable

- Org-wide AI collaboration

Developer Workflow



Team starts
with a
product
spec.



AI agent
drafts
boilerplate,
writes tests,
scaffolds
APIs.



Human
edits high-
level logic.



Verification
pipeline
catches
regressions.



Persistent
memory
tracks
decisions.

Organisation Implications

- **How team roles change:**
 - less boilerplate, more design/strategy.
- **Challenges:**
 - trust, code quality, AI oversight, onboarding.
- **What's emerging:**
 - AI-augmented org charts, agent-driven sprints.

